

Real Time Object Detection and Tracking using Yolov8 and Deep Sort

Chandan Kumar K R¹, Kiran H R², Gopinath Ramaje³, B E Students, Dept Computer Science and Engineering,
RV college of engineering Bangaluru, Karnataka, India

Abstract- Over the past two decades, computer vision has gained prominence, with visual object tracking becoming a critical area of research. Object tracking involves continuously monitoring the movement of objects over time, addressing challenges such as tracking multiple objects within a frame. This research proposes a three-stage approach: detecting, identifying, and tracking objects within specific zones. While advancements have been made, this study introduces the use of Kalman filters to enhance tracking accuracy. The YOLO algorithm is employed for object detection and recognition, followed by motion prediction and feature generation through Kalman filters to model the states of moving objects. The Deep SORT algorithm is then used for tracking, combining a deep association metric with the SORT algorithm. Kalman filters further improve the model's performance in object tracking. The goal is to accurately detect or associate target objects across consecutive video frames. This paper analyzes the tracking-by-detection approach, using YOLO for detection and SORT for tracking. A custom image dataset, trained on six specific classes using YOLO, is utilized in the videos, with SORT used for tracking. Real-time recognition of vehicles or pedestrians in video streams has significant applications in traffic analysis. This paper provides a comprehensive analysis and serves as a valuable knowledge resource in the domain of visual object tracking.

Keywords- Tracking-by-detection, You Only Look Once (YOLO), Simple Online and Realtime Tracking (SORT), visual tracking.

I. INTRODUCTION

Video tracking is aimed at associating target objects in consecutive video frames. When the objects move quickly relative to the frame rate, the association can be particularly difficult. Another situation that increases the problem's complexity is when the object being tracked changes orientation over time.

In an object detection and tracking system, the algorithm processes sequential video frames to identify and track objects over time. Different algorithms, such as YOLO for detection and Deep SORT for tracking, offer unique advantages and limitations. When choosing the appropriate algorithms, it's crucial to align them with the specific application requirements. The object detection and tracking system comprises two key components: object identification and localization, along with tracking and data management for analysis and reporting. Object tracking [4] involves a computer vision task that extracts the movement of an object across a sequence of images, predicting its trajectory. Object detection and tracking are two interconnected processes: Detection identifies the presence of an object, asking the question.

"Is something there?" while tracking focuses on predicting the object's path, asking, "Where is it going?". Visual tracking is a challenging task in computer vision due to factors such as target deformations, variations in lighting, changes in scale, rapid and abrupt movements, partial occlusions, motion blur, object

deformations, and background clutter. Recent advancements in object detection techniques have led to the development of various tracking-by-detection methods. The goal of this project is to detect and track objects in a video using a tracking-by-detection approach.

Classification: Given an image patch, determine which of the predefined categories is present in that patch.

Localization and Detection: Given a complex image, identify whether a specific object of interest is present within the image and provide precise location details of that object [.

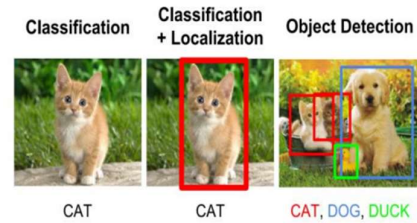


Fig -1: Object Detection Process

II. LITERATURE SURVEY

Object detection and tracking is a fundamental problem in computer vision, with applications ranging from autonomous vehicles to surveillance systems. It involves identifying objects within an image or video sequence and tracking their movement over time. Recent advances in deep learning have significantly improved the accuracy and efficiency of object detection models, such as the YOLO [13] (You Only Look Once) family, which can detect and classify objects in real-time. However, tracking these objects over time remains a challenging task due to factors like occlusion, variations in lighting, and fast motion.

Object detection has evolved through various stages, starting from traditional methods like edge detection and feature extraction to modern deep learning-based approaches. The introduction of convolutional neural networks (CNNs) has been a game changer, enabling the development of models like YOLO, SSD (Single Shot Multi-Box Detector), and Faster R-CNN [5]. These models can detect multiple objects within a single image, making them suitable for real-time applications. YOLO, in particular, is known for its speed and accuracy, making it a popular choice for many applications.

Neural networks are a class of algorithms developed to identify patterns, inspired by the structure and function of the human brain. They process sensory information through a method akin to raw input perception, performing tasks such as face detection, identifying individuals in images, and recognizing facial expressions. They can also detect and classify objects in

photos or videos. The process involves clustering and grouping similar features, with deep learning models establishing connections, such as between pixels in an image [20].

Object tracking involves estimating the trajectory of detected objects as they move across frames in a video. Traditional tracking methods include algorithms like Kalman filters, mean-shift, and optical flow. With the advent of deep learning, tracking-by-detection has gained popularity, where detected objects in each frame are linked across frames using data association techniques. Deep SORT (Simple Online and Realtime Tracking) is one such method that builds upon the SORT algorithm by incorporating appearance information, allowing for more robust tracking in challenging scenarios. Most researchers use techniques of deep learning to extract qualified deep characteristics. Object Detection [4] is modelled as a classification problem where at all possible locations we take gaps of fixed sizes from the input object to feed these patches into an image classifier. Every window is fed to the classifier which determines the object's class in the window. Therefore, we know the category and location of the image objects. Traditional object detection algorithms are divided into three steps: selecting the detection windows, extracting features of the windows, and designing a classifier [23].

Application:

Object detection is making significant inroads across various industries, with applications extending from personal security to workplace productivity. This technology plays a crucial role in numerous areas of computer vision, such as image retrieval, security, surveillance, autonomous vehicle systems, and machine inspection. Despite its advancements, object recognition still faces substantial challenges. The potential applications for object detection are vast, with exciting opportunities for future developments. In the following sections, we will explore some of the current and emerging use cases in detail [18].

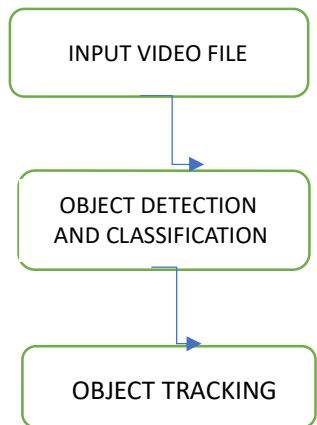


Fig -1: Basic flow diagram for object detection and tracking

Convolutional Neural Networks (CNNs) are composed of neurons with learnable weights and biases, similar to traditional neural networks. Each neuron processes multiple inputs by calculating a weighted sum, applying an activation function, and producing an output. CNNs are frequently utilized to identify patterns such as edges (both vertical and horizontal),

shapes, colors, and textures in object detection tasks. A typical CNN architecture might include layers in the sequence: [INPUT — CONV — RELU — POOL — FC]. Over the years, numerous algorithms for object detection have been developed.

Since the introduction of R-CNN, several enhanced models have been developed, including Fast R-CNN, which simultaneously optimizes classification and bounding box regression tasks, Faster R-CNN [15], which incorporates an additional subnetwork to generate region proposals, and YOLO

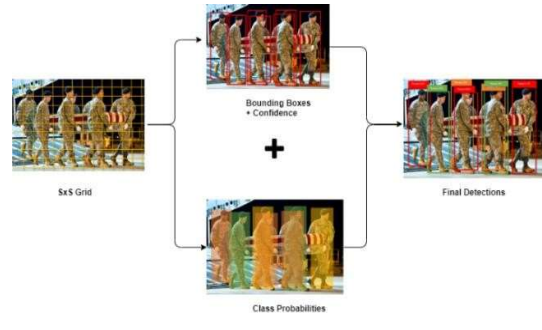


Fig -3: YOLOv8 on custom database

[1], which performs object detection using a fixed grid regression approach. Each of these advancements has contributed to improving detection efficiency compared to the original R-CNN, making real-time object recognition more practical and accurate [6].

YOLO is one of the fastest algorithms available for object detection. While it may not be the most accurate algorithm, it is an excellent choice when real-time detection is required without significant loss of precision. YOLO operates by using a single convolutional network to simultaneously predict multiple bounding boxes and category probabilities for these boxes [1].

In the field of computer vision, object tracking plays a crucial role. Object trackers can be classified into two main categories: TBD (Tracking by Detection) and DFT (Detection-Free Tracking). Additionally, trackers can be divided into online and offline types, depending on whether future frames are utilized [4]. Object tracking involves the process of following an object across a sequence of frames, which could be a person, a ball, or a vehicle. The process begins by identifying all potential detections in a frame and assigning each detection a unique ID. In subsequent frames, the algorithm attempts to maintain the same object ID as the object moves. If an object exits the frame, the ID is removed, and if a new object appears, a new ID is assigned. This task is challenging due to various factors, such as objects looking similar, causing ID switches, objects becoming occluded (e.g., when something is hidden behind another object), or objects disappearing and reappearing later.

III. METHODOLOGY

Object Detection and Tracking, integral components of Deep learning and computer vision, are utilized and recognize and monitor instances of visual objects across various categories within their surroundings. Over time, these technologies have advanced significantly in terms of algorithms and architectures, with notable progress in Object Detection during 2010s and in

Object Tracking throughout the 2020s. These methods have already found applications in a wide range of fields [24].

The approach utilized for visual object tracking in videos involves the integration of object detection using YOLO [1] and tracking through the SORT algorithm [2]. The entire implementation is carried out using Python.

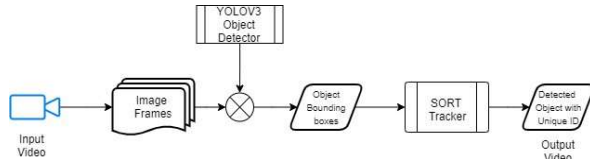


Fig -2: Flowchart representation of Visual Object Detection and

In this method, an input video is processed by first extracting the total number of frames, which are then passed to the object detector, YOLO. As an object detector, YOLO generates bounding boxes with class IDs and confidence scores for each detected object [1]. Following the tracking-by-detection approach, these detections are subsequently fed into the SORT tracker [2] for continuous object tracking throughout the video. SORT is an online tracker that operates on the tracking-by-detection principle. This method relies on a robust detector to identify objects and Kalman filter to track them. SORT tracks each detection by assigning a unique ID to each bounding box. If an object is lost due to occlusion, misidentification, or other reasons, the tracker assigns a new ID and begins tracking the newly detected object [10].

The Streamlit web app developed here is designed to perform real-time object detection and tracking using the YOLOv8 model and storing the detected objects in a SQLite database. It allows users to upload images, videos, or use a webcam to detect and track objects, and it provides a user-friendly interface to interact with the detection and tracking system. Here's a breakdown of how the app works:

1. Environment Setup - Importing Libraries:

The app imports essential libraries, including 'Pathlib' for managing file paths, 'cv2' for image processing, 'ultralytics' for YOLO-based object detection, and 'sqlite3' for database management. 'Pathlib' is used to dynamically handle file paths, allowing the app to determine the absolute path of the current file and to set up directories for images, models, and databases efficiently. These components are crucial for the app's functionality, ensuring smooth processing, detection, and data management.

2. Model Configuration- YOLO Model Loading:

The YOLOv8 model ('yolov8n.pt') is loaded from the specified directory. This model is used for object detection tasks in images, videos, and webcam streams.

3. Database Initialization- SQLite Database:

The app sets up a SQLite database ('object_detections.db') to store information about detected objects. The database schema includes fields such as 'id', 'object name', 'count', 'date', and 'time' to

organize the data. A table named 'detections' is created if it doesn't already exist, providing a structured way to record and manage data related to object detection.

4. Streamlit Interface Setup- Custom CSS:

Custom CSS is employed to define the width of Streamlit components, ensuring a clean and organized appearance for the app. The app features a title, "Object Detection and Tracking," prominently displayed at the top, with a horizontal separator beneath it to maintain a tidy and user-friendly interface.

5. User Interaction Components- Confidence Slider:

Users can adjust the confidence level of the YOLO model using a slider, which allows them to control the sensitivity of the detection algorithm. Additionally, users can choose the source of input from a dropdown list, with options including 'Image', 'Video', and 'Camera', providing flexibility in how the data is fed into the app.

6. Handling Different Sources

Image Upload:

Users can upload images in JPEG or PNG format. Once uploaded, the image is displayed in the app, and the YOLO model processes it to detect objects. The app displays the processed image with bounding boxes around the detected objects. Additionally, the details of these detected objects are saved to the database for further analysis and record-keeping.

Video Upload:

Users can upload video files in formats such as MP4, AVI, MOV, or MKV. The app processes the video frame by frame using the YOLO model to detect objects, displaying each processed frame in real-time with highlighted detections. After the video processing is complete, the detected objects are accumulated and saved to the database.

Webcam Stream:

Users can activate their webcam to capture real-time video, which is then processed on-the-fly to detect objects. The detected objects from the live webcam stream are continuously saved to the database. A checkbox is provided for users to stop the webcam stream whenever they choose.

Phone Streaming:

Users can stream video from their phone using apps like IP Webcam (Android) or EpocCam (iOS). After connecting the phone and computer to the same Wi-Fi network and entering the stream URL in the app, the video feed from the phone is captured and processed by the YOLO model. Detected objects are highlighted in the live stream and continuously saved to the database. If the stream encounters issues, such as an invalid URL, the app provides warnings and prompts the user to check the connection.

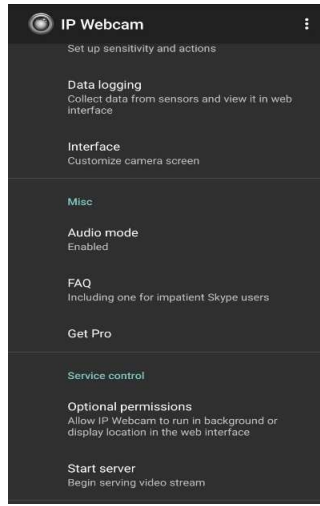


Fig -4: Ip webcam Interface

7.Database Interaction

Saving Detected Objects: The app stores detected objects in the database, recording the object name, count, detection date, and time.

Displaying Detected Data: The app retrieves and displays this data in a table format, providing users with a comprehensive history of all detected objects.

8. Backend Processing

Frame Processing:

Each frame from the image, video, or webcam stream is processed by the YOLO model, which returns bounding boxes, class IDs, and confidence scores for the detected objects. The frame is then annotated with bounding boxes and labels indicating the detected objects. These detected objects are collected in a list for further processing and storage.

9. Streamlit Application Execution

The app is designed to respond dynamically to user inputs, executing the appropriate logic based on the selected source—whether it be an image, video, or webcam. The results are then displayed on the Streamlit interface, while the detected data is simultaneously saved to the database for record-keeping.

This app can be extended further to include more advanced features like tracking, more detailed analytics on detected objects, and integration with other data sources.

IV. Result

The proposed system was tested on multiple video sequences, each presenting different challenges such as varying object sizes, rapid object movement, and varying lighting conditions. The performance of the system was evaluated based on detection accuracy, tracking stability, and processing speed (FPS).

The system was able to accurately detect and classify objects in the video sequences. Using the YOLOv8 model with a confidence threshold of 85%, the system successfully identified

multiple objects including pedestrians, vehicles, and other common items. The detection accuracy was consistent across different video sequences, demonstrating the robustness of the YOLOv8 model.

The integration of the SORT algorithm provided effective tracking by maintaining the identity of detected objects across consecutive frames. Even with occlusions and rapid movements, SORT managed to re-identify objects once they reappeared in the frame. However, in cases of prolonged occlusion or when objects exited and re-entered the frame, the algorithm assigned a new ID, which is a known limitation of SORT. Despite this, the overall tracking performance was stable and reliable.

The system's processing speed was evaluated by calculating the frames per second (FPS) during video processing. On average, the system achieved an FPS rate of approximately 18-25 frames per second, depending on the complexity of the scene and the number of detected objects. This speed is sufficient for real-time applications, allowing for smooth video playback while maintaining detection and tracking accuracy.

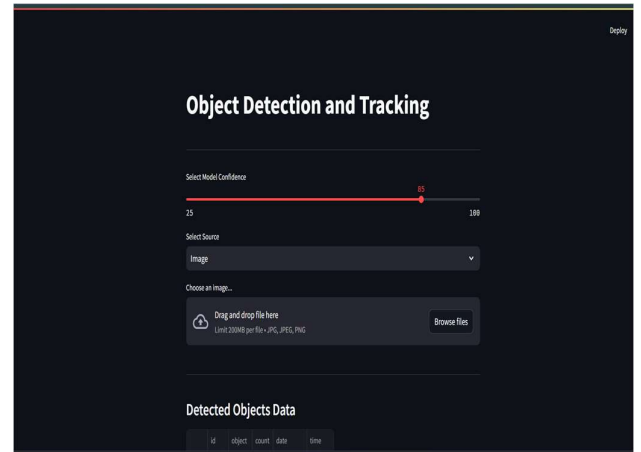


Fig 5. Stream-lit web app Interface

During the visual object detection and tracking task, the input video is divided into individual frames. Each frame undergoes processing where object detection is performed using the YOLO model, and object tracking is managed by the SORT algorithm. Both the processed frames and the complete video output are saved, including the detection and tracking information.

The detection information includes bounding boxes around detected objects, along with the class names and confidence scores provided by YOLO [25]. Additionally, SORT assigns a unique ID to each detected object, which is displayed on the output screens.

Below are examples of the output screens from the videos tested, illustrating the bounding boxes with class names, confidence scores, and unique IDs assigned by SORT.

This explanation provides a clear understanding of how the detection and tracking task is performed, including the roles of YOLO and SORT, as well as the type of output generated.

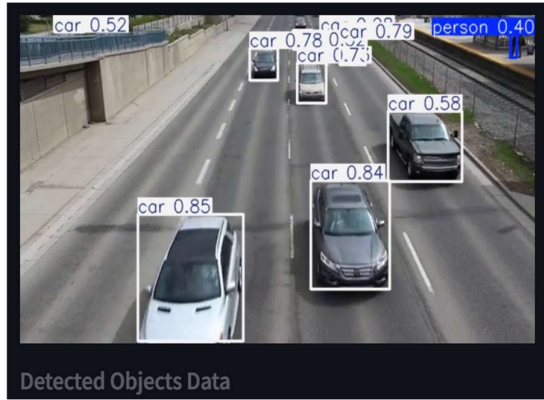


Fig -6: Objects detected in ongoing videos

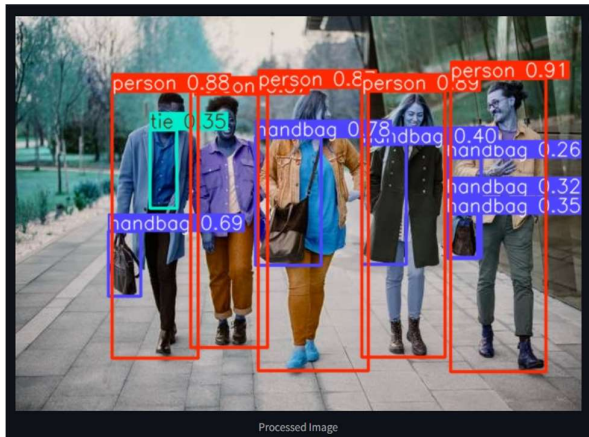


Fig -7: Objects detected in image

V. Discussions

The results obtained from testing the proposed system across multiple video sequences indicate promising performance in object detection, classification, and tracking. The combination of the YOLOv8 model and the SORT algorithm proved effective in handling the challenges posed by varying object sizes, rapid movements, and fluctuating lighting conditions. However, while the system demonstrated robustness and reliability, several aspects warrant further discussion to contextualize these findings within the broader scope of object detection and tracking.

Detection Accuracy and Robustness

The YOLOv8 model, with a confidence threshold set at 85%, consistently detected and classified a variety of objects, including pedestrians, vehicles, and other common items across diverse video sequences. The robustness of YOLOv8 was evident in its consistent performance despite challenges such as varying object sizes and lighting conditions. This reinforces the model's suitability for real-time applications where reliability across different environments is crucial. However, it is worth considering that the choice of a high confidence threshold, while reducing false positives, may have also led to the omission of less prominent objects. A comparative analysis with lower thresholds could provide insights into the trade-offs between detection accuracy and sensitivity.

Tracking Stability with SORT Algorithm

The integration of the SORT algorithm contributed significantly to the system's ability to maintain object identity across consecutive frames. The algorithm's strength lies in its ability to handle occlusions and rapid movements, allowing it to re-identify objects effectively when they reappear in the frame. This feature is particularly valuable in dynamic environments where objects frequently interact or move unpredictably.

However, the limitations of SORT were also observed. Specifically, the algorithm's tendency to assign new IDs to objects after prolonged occlusion or when objects exit and re-enter the frame is a known challenge. This issue could potentially lead to inconsistencies in tracking over extended sequences, especially in crowded or complex scenes. Future work could explore the integration of more advanced tracking algorithms, such as Deep SORT, which incorporates appearance information, to enhance re-identification capabilities and mitigate this limitation.

Processing Speed and Real-Time Application

The system's processing speed, measured in frames per second (FPS), ranged from 18 to 25 FPS, depending on scene complexity and the number of detected objects. This performance is sufficient for real-time applications, ensuring smooth video playback while maintaining detection and tracking accuracy. However, the observed variation in FPS highlights the impact of scene complexity on processing speed. In scenarios with a high density of objects or significant motion, the system's FPS tends to decrease, which could affect real-time performance. Optimizing the computational efficiency of the system, perhaps through model pruning or hardware acceleration, could help maintain a more consistent FPS across various scenarios.

Future Work and Considerations

While the current system demonstrates robust performance in object detection and tracking, several areas for future improvement and exploration remain. Enhancing the tracking algorithm to better handle prolonged occlusions and object re-entries could improve the overall reliability of the system. Additionally, further optimizing the processing pipeline to maintain consistent FPS across different scenes would enhance its applicability in more demanding real-time scenarios.

Moreover, extending the system's capabilities to include additional tasks such as object counting, behaviour analysis, or anomaly detection could provide valuable functionalities for specific applications. Integrating a database to store detection and tracking results would also facilitate long-term analysis and provide a foundation for more advanced analytics.

VI. Conclusion

The project successfully implements a real-time object detection and tracking system using the YOLO algorithm for detection and the SORT algorithm for tracking. By integrating these advanced techniques into a Streamlit web application, the project provides an intuitive interface for users to detect and

track objects across various media, including images, videos, and live camera feeds. The application effectively demonstrates the capabilities of YOLO's high-speed object detection combined with the robust tracking capabilities of SORT, resulting in a system that can handle real-time object tracking with remarkable accuracy. The integration of a SQLite database allows for efficient storage and retrieval of detected object data, providing a valuable resource for analysing trends and patterns over time. The flexibility of the system, allowing for customization of confidence levels and seamless handling of different media sources, further enhances its usability and adaptability to various applications.

VI. Bibliography

- [1] D. J. Redmon and A. Farhadi, "Yolov3: An incremental improvement," arXiv preprint arXiv:1804.02767, 2018.
- [2] A. Bewley, Z. Ge, L. Ott, F. Ramos and B. Upcroft, "Simple online and realtime tracking," 2016 IEEE International Conference on Image Processing, Phoenix, AZ, 2016, pp. 3464-3468.
- [3] S. Moon, J. Lee, D. Nam, H. Kim and W. Kim, "A comparative study on multi-object tracking methods for sports events", 2017 19th International Conference on Advanced Communication Technology (ICACT), 2017.
- [4] A. Mekonnen and F. Lerasle, "Comparative Evaluations of Selected Tracking-by-Detection Approaches," IEEE Transactions on Circuits and Systems for Video Technology, vol. 29, no. 4, pp. 996-1010, 2019.
- [5] S. Nayak, "Training YOLOv3: Deep Learning based Custom Object Detector | Learn OpenCV," Leamopencv.com, 2019. [Online]. Available: <https://www.leamopencv.com/training-yolov3-deep-learning-based-custom-object-detector/>.
- [6] S. Shinde, A. Kothari and V. Gupta, "YOLO based Human Action Recognition and Localization," Procedia Computer Science, vol. 133, pp. 831-838, 2018.
- [7] L. Liu et al., "Deep Learning for Generic Object Detection: A Survey," International Journal of Computer Vision, 2019.
- [8] Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi. "You Only Look Once: Unified, Real-Time Object Detection". 779-788. 10.1109/CVPR.2016.91 (2016).
- [9] Dillon Reis, Jordan Kupec, Jacqueline Hong, Ahmad Daoudi, "Real-Time Flying Object Detection with YOLOv8", arXiv:2305.09972v1 [cs.CV] 17 May 2023
- [10] Nicolai Wojke, Alex Bewley, Dietrich Paulus, "Simple Online and Realtime Tracking with a Deep Association Metric", arXiv:1703.07402v1 [cs.CV]
- [11] P. SHARMA, "A Step-by-Step Introduction to the Basic Object Detection Algorithms (Part 1)," Analytics Vidhya, 2019. [Online]. Available: <https://www.analyticsvidhya.com/blog/2018/10/a-step-by-step-introduction-to-the-basic-object-detection-algorithms-part-1/>.
- [12] Zhao, Z.Q., Zheng, P., Xu, S.t., Wu, X., "Object Detection with Deep Learning: A Review," arXiv:1807.05511v1 [cs.CV] 15 Jul 2018
- [13] Jacob Solawetz and Francesco. "What is yolov8? the ultimate guide", 2023.
- [14] "Object Tracking in Deep Learning - MissingLink.ai," MissingLink.ai, 2019. [Online]. Available: <https://missinglink.ai/guides/computer-vision/object-tracking-deep-learning/>.
- [15] D. Parthasarathy, "A Brief History of CNNs in Image Segmentation: From R-CNN to Mask R-CNN," Medium, 2019. [Online]. Available: <https://blog.athelas.com/a-brief-history-of-cnns-in-image-segmentation-from-r-cnn-to-mask-r-cn34ea83205de4>.
- [16] Chinthakindi Kiran Kumar, Kirti Rawal, "A Brief Study on Object Detection and Tracking", J. Phys.: Conf. Ser. 2327 012012, 2022.
- [17] D.Venkata Siva Prasad, Ch.Niharika, "Object Detection and Tracking using Image Processing", IJCRT, Volume 8, ISSN: 2320-28820, 6 June 2020
- [18] Abdul Vahab, Maruti S Naik, Prasanna G Raikar, "Applications of Object Detection System". IRJET, e-ISSN: 2395-0056, Volume: 06 Issue: 04, April 2019
- [19] Amandeep, Er Monica Goyal, "Moving Object Detection Techniques". Int. J. Comput. Sci. Mob. Comput. 4, 345 – 349, 2015
- [20] Akansha Bathija, Grishma Sharma, "Visual Object Detection and Tracking using YOLO and SORT", IJERT Vol. 8 Issue. ISSN: 2278-0181, November-2019
- [21] Shubham Shinde, Ashwin Kothari, Vikram Gupta. "YOLO based Human Action Recognition and Localization." Procedia Computer Science. 133. 831-838. 10.1016/j.procs.2018.07.112. (2018)
- [22] Yadav, A.; Chaturvedi, P.K.; Rani, S. Object Detection and Tracking using YOLOv8 and DeepSORT.; Tripathi, A.K., Shrivastava, V., Eds.; Computing and Intelligent Systems; SCRS: New Delhi, India, 2023; pp. 81–90.
- [23] Xu, X.C.; Liu, Y.; Lyu, L.; Yan, P.; Zhang, J.Y. MAD-YOLO: A quantitative detection algorithm for dense small-scale marine benthos. *Ecol. Inform.* **2023**,
- [24] Ariza-Sentis, M.; Velez, S.; Martínez-Pena, R.; Baja, H.; Valente, J. Object detection and tracking in Precision Farming: A systematic review. *Comput. Electron. Agric.* **2024**
- [25] N. M. Krishna, R. Y. Reddy, M. S. C. Reddy, K. P. Madhav and G Sudham, Object Detection and Tracking Using Yolo, 2021.